CIS 3300 · THE SPEEDRUN LEADERBOARD

Getting Started

A running example, built one checkpoint at a time

Speedrunners compete to finish a video game as fast as possible, and a leaderboard records each run: the player, the game and category, the time, the date, and whether moderators verified it. In these guides we build a program that reads submitted runs from a file, rejects the bad ones, and reports world records, personal bests, and rankings.

We build it in class, one checkpoint at a time, from Week 12 to Week 14, and these guides contain every step, including any we do not reach. Each checkpoint prepares you for part of Lab W14, which repeats the same process with weather data but different details, so you need to understand each step, not copy it.

To see a real leaderboard, visit `https://www.speedrun.com/sm64`, which includes the 16 Star and 120 Star categories used in our data. Our game titles and categories are real, but the players and times are invented.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

1. How to Use These Guides

Each checkpoint is a complete Eclipse project that compiles and runs, and each checkpoint has a guide of its own. Every checkpoint guide starts from one project and ends at the next, as Table 1 shows. That means you can start any checkpoint, even if you missed the class before it: import the project its guide starts from, and work through the steps.

Table 1: The Eclipse projects, and where each one is used.

Project	Use it to
`SpeedrunStarter`	Start the Starter Program, below, and Checkpoint 1
`SpeedrunCheckpoint1`	Check your Checkpoint 1 work, or start Checkpoint 2
`SpeedrunCheckpoint2`	Check your Checkpoint 2 work, or start Checkpoint 3
`SpeedrunCheckpoint3`	Check your Checkpoint 3 work, or start Checkpoint 4
`SpeedrunCheckpoint4`	Check your Checkpoint 4 work, or start Checkpoint 5
`SpeedrunCheckpoint5`	Check your Checkpoint 5 work, or start Checkpoint 6
`SpeedrunCheckpoint6`	Check your Checkpoint 6 work: the finished program

Importing a project into Eclipse

1. Download the project's zip file. You do not need to unzip it.
2. In Eclipse, choose **File**, then **Import**. Open **General**, choose **Existing Projects into Workspace**, and click **Next**.
3. Choose **Select archive file**, browse to the zip file, make sure the project is checked, and click **Finish**.

Every project has the same layout: a `speedrun` package for the classes that represent the data, an `app` package for the program with `main`, and a `files` folder for the data files.

How each checkpoint guide works

Each checkpoint guide lists the project it starts from, the project it ends at, and the Lab W14 tasks it prepares you for. Then it gives numbered steps. Each step shows the code to type and the output you should see when you run it. If your output does not match, stop and compare your code with the listing before going on, because each step builds on the one before it. Each guide ends with *Check yourself* questions and their answers.

2. The Data

The file `files/runs.csv` holds one submitted run per line, after a header line. The runs are in the order they were submitted, not in any useful order:

```
player,game,category,time,date,verified
NovaByte,Super Mario 64,16 Star,16:51.5,2025-09-03,no
Kestrel,The Legend of Zelda: Ocarina of Time,100%,4:05:54.7,2025-11-11,no
```

Times are written the way speedrunners write them: minutes, seconds, and tenths, such as `16:51.5`, or hours first when a run takes an hour or more, such as `4:05:54.7`.

Not every line is good. Table 2 lists what the leaderboard accepts. A line that does not have exactly six fields, or has a field the table does not accept, is a *bad line*. The file `files/runs_clean.csv` has eight good runs and no bad lines, for testing.

Table 2: What the leaderboard accepts in each field.

Field	Accepted values
`player`, `game`, `category`	Text that is not blank
`time`	A time written as `M:SS.s` or `H:MM:SS.s`, with minutes and seconds below 60, and greater than 0
`date`	A real date written as year, month, and day, such as `2026-02-14`, and not in the future
`verified`	Exactly `yes` or `no`

3. The Starter Program

Starts from: SpeedrunStarter. **Changes nothing:** this section only runs it.

The starter program reads the clean file, splits each line on commas, turns the time and the date into Java objects, and prints each run. Here is all of it:

```
public class SpeedrunReport {

    public static void main(String[] args) throws IOException {
        List<String> lines = Files.readAllLines(Path.of("files/runs_clean.csv"));

        // Line 0 is the header, so the runs start at line 1.
        for (int i = 1; i < lines.size(); i++) {
            String[] fields = lines.get(i).split(",");
            String player = fields[0];
            String game = fields[1];
            String category = fields[2];
            Duration time = RunTime.parse(fields[3]);
            LocalDate date = LocalDate.parse(fields[4]);
            boolean verified = fields[5].equals("yes");

            System.out.printf("%-10s %-36s %-8s %10s  %s  %s%n",
                    player, game, category, RunTime.format(time), date,
                    verified ? "verified" : "unverified");
        }
    }
}
```

Three things in it may be new:

- `Files.readAllLines` reads the whole file into a `List<String>`, one element per line.
- `Duration` is Java's class for an amount of time, in the `java.time` package, next to `LocalDate`.
- `RunTime` is a small class in the `speedrun` package that is provided for you. `RunTime.parse` turns text such as `16:51.5` into a `Duration`, and `RunTime.format` turns a `Duration` back into text. Use it as it is.

Step 1: Run it on the clean file

Import SpeedrunStarter, open `SpeedrunReport.java` in the `app` package, and run it.

```
NovaByte   Super Mario 64                      16 Star     16:51.5  2025-09-03  unverified
Tangent    Super Mario 64                      16 Star     15:42.6  2025-09-24  unverified
Kestrel    The Legend of Zelda: Ocarina of Time 100%      4:05:54.7  2025-11-11  unverified
Lumen      Super Mario 64                      16 Star     17:03.4  2025-11-15  verified
Lumen      Super Mario 64                      16 Star     17:06.4  2025-12-13  verified
NovaByte   Super Mario 64                      120 Star  1:43:33.5  2025-12-19  verified
Moss       The Legend of Zelda: Ocarina of Time 100%      3:50:02.6  2025-12-25  unverified
Sprocket   Super Mario 64                      120 Star  1:38:42.1  2025-12-28  verified
```

Step 2: Run it on the real file

Change `runs_clean.csv` to `runs.csv` in the file name, and run it again.

```
NovaByte   Super Mario 64                        16 Star     16:51.5  2025-09-03  unverified
Tangent    Super Mario 64                        16 Star     15:42.6  2025-09-24  unverified
Kestrel    The Legend of Zelda: Ocarina of Time 100%      4:05:54.7  2025-11-11  unverified
Lumen      Super Mario 64                        16 Star     17:03.4  2025-11-15  verified
Lumen      Super Mario 64                        16 Star     17:06.4  2025-12-13  verified
Exception in thread "main" java.lang.NumberFormatException: For input string: "4x.1"
        at java.base/jdk.internal.math.FloatingDecimal.readJavaFormatString(FloatingDecimal.java
    :2054)
        at java.base/jdk.internal.math.FloatingDecimal.parseDouble(FloatingDecimal.java:110)
        at java.base/java.lang.Double.parseDouble(Double.java:792)
        at speedrun.RunTime.parse(RunTime.java:38)
        at app.SpeedrunReport.main(SpeedrunReport.java:30)
```

Read a stack trace from the top. The first line says *what* went wrong: a `NumberFormatException`, because `"4x.1"` is not a number. The lines below it say *where*, starting with the method that threw it and ending with `main`. Skip the lines inside `java.base`, which are Java's own code, and look for the first line in code from this project. Here, `RunTime.parse` threw the exception while `main` was calling it on line 30.

The bad line is line 7 of the file:

```
Kestrel,Celeste,Any%,28:4x.1,2025-10-02,yes
```

Someone typed an `x` instead of a digit. The real problem is what happened next: nothing. The program stopped, and the 36 good runs after that line were never read. One typo threw away the whole leaderboard.

Step 3: Run it with a missing file

Change the file name to `run_clean.csv`, without the `s`, and run it again.

```
Exception in thread "main" java.nio.file.NoSuchFileException: files/run_clean.csv
        at java.base/sun.nio.fs.UnixException.translateToIOException(UnixException.java:92)
        ... (ten more lines inside java.base)
        at app.SpeedrunReport.main(SpeedrunReport.java:22)
```

The real trace has ten more lines of Java's own code where the `...` is; your line numbers inside `java.base` may also differ, depending on your version of Java. The message itself is useful, because it names the file, but a person running the program should never see a stack trace. Change the file name back to `runs_clean.csv`.

Next: the Checkpoint 1 guide, which starts from SpeedrunStarter.